



Ctags and Cscope

In the last article on Ctags, I discussed how to install and use Ctags to browse source code. In this article, I do the same for Cscope. This article also includes a comparison between Cscope and Ctags.

Cscope is an interactive tool to browse C source code. It examines *.c* and *.h* files, *yacc* (*.y* files) and *lex* (*.l* files) in the current directory. It maintains a symbol cross-reference (*cscope.out*), which is used for finding language objects including C pre-processor symbols, function calls, function declarations, etc. The symbol table for cross-reference is created only once, when Cscope is used on the source files for the first time; it is rebuilt when the source is changed.

Features

Cscope creates its own database to allow faster searching. It provides a number of options for searching source code; it allows you to search for global definitions, calling functions, called functions, any C symbol, and any text string. It also enables you to find files, and to find files that *#include* a particular file. It even lets you change a text string.

```
/* Return gcc_builtin_vector (outer mode, inner mode) */
else
    return result_s;
}

/* Simplify SUBREG:OUTERMODE(OP:INNERMODE, BYTE)
   Return 0 if no simplifications are possible. */
rtx
simplify_subreg (enum machine_mode outermode, rtx op,
                 enum machine_mode innermode, unsigned int byte)
{
    /* Little bit of sanity checking. */
    gcc_assert (innermode != VOIDmode);
    gcc_assert (outermode != VOIDmode);
    gcc_assert (innermode != BLKmode);
    gcc_assert (outermode != BLKmode);

    gcc_assert ((GET_MODE (op) == innermode
                 || GET_MODE (op) == VOIDmode);
                ((byte < GET_MODE_SIZE (outermode)) == 0);
    gcc_assert (byte < GET_MODE_SIZE (innermode));

    if (outermode == innermode && !byte)
        return op;
}
```

Figure 1: Definition of symbol or function in a particular file

```
for (i = 0; i < size; i += subsize)
{
    rtx partial = simplify_subreg (mode, x[mode], i);
    if (!partial || !assembler_integer (partial, subsize, subalign, 0))
        break;
    if (i == size)
        return true;
    /* If we've printed some of it, but not all of it, there's no going
       back now. */
    gcc_assert (!i);
}

gcc_assert (!force);
return false;
}
```

Figure 2: Calling function pointing to a function call

Installing Cscope

To install Cscope, you need to compile from the source. Get the latest version from <http://sourceforge.net/projects/cscope/files/> and follow the instructions given below.

 **Note:** The steps to build and configure Cscope on Linux and Cygwin are the same. I built and installed Cscope under Cygwin on Windows.

Go to the *Downloads* folder and extract the source with `tar -xvf cscope-15.7a.tar.bz2`.

Make a separate directory, *build_scope*, with `mkdir build_scope`. Change into the *build_scope* directory, and then *configure* the source by running the configure utility in the extracted source. For example: `/cygdrive/c/Users/root/Downloads/cscope-15.7a/configure`.

To build Cscope, next run *'make'*, followed by (as the root user) *make install* to install it. Check if the *cscope* command is available (on Windows+Cygwin, look for *cscope.exe* in the folder `'/cygwin/usr/local/bin'`). If it is, the build was successful.

Using Cscope

Change to the directory containing the files you want to index for search. In this example, let's go to the GCC source code with `cd /cygdrive/d/source_gcc/gcc-4.7.0/gcc/`. To generate the Cscope symbol table for cross reference, run `cscope -R *` and wait (it takes time to build the database). Then press the *Return* key to continue with Cscope. Your terminal will show a list of various functions that can be performed by Cscope.

The following are some examples that show the various functions of Cscope.

To find a global definition, enter the name of the symbol or function against the relevant option. I tried to find the function *'simplify_subreg'*. The results are listed, and to choose from them, go to the particular option and press *Enter*. This will redirect *yoy* to the particular file containing that symbol or function definition, as shown in Figure 1. Now, *:q* is used to quit from the file containing the function or symbol definition; *CTRL+D* is used to quit